

CUDA Warp Specialization Project

Saswata Mishra, Suyash Chowdhury, Rajat Maheshwari

November 2025

1 Problem Statement

Modern GPUs provide extremely high computational throughput, yet many real-world workloads fail to reach peak performance due to the fundamental imbalance between computation and memory operations. Global memory accesses incur high latency, and when threads are responsible for both computation and data movement, these long-latency operations stall the compute pipeline. As a result, kernels with even moderate arithmetic intensity often achieve only a fraction of the available FLOP/s because memory stalls, insufficient memory-level parallelism (MLP), and poor overlap between data transfers and computation limit effective utilization of the hardware.

Warp specialization has emerged as a promising technique to address this bottleneck. Instead of assigning identical responsibilities to all warps within a thread block, warp specialization divides them into *memory warps*—dedicated to fetching, staging, and writing data—and *compute warps* that exclusively perform arithmetic operations on already available on-chip data. This decoupling enables the GPU to overlap global memory transfers with computation, reduce pipeline stalls, improve coalescing, increase instruction-level parallelism (ILP), and more fully exploit shared memory as a software-managed cache. Early frameworks such as CudaDMA demonstrated that specialized DMA warps, combined with fine-grained named-barrier synchronization, can significantly improve bandwidth utilization and produce speedups of $1.15\times$ – $3.2\times$ on representative HPC kernels.

This project aims to extend and generalize these ideas by designing, implementing, and evaluating new warp-specialization strategies that maximize compute–memory overlap across diverse workloads. We explore multiple kernel types (FP32, mixed FP32/FP64, and Tensor Core variants), apply specialization techniques to several PolyBench and Rodinia benchmarks, and systematically evaluate their performance using NVIDIA profiling tools. The goal is to characterize when warp specialization is beneficial, identify optimal transfer/computation scheduling patterns, and derive generalizable design principles for future GPU kernel developers.

2 Case Study - GEMM kernel

1. **Baseline Kernel** - Square tiling.
2. **Baseline Kernel 2** - Rectangular tiling.
3. **Miscellaneous optimizations** - Double buffering using indexed loads, Double buffering using `cp.async`.
4. **Warp Specialization** - Manual implementations using branching.
5. **Optimal/Super Optimal** - Wider memory load instructions, loop unrolling for batched load instructions, register tiling.
6. **CudaDMA** - Warp specializations using cudaDMA library.
7. **Custom scheduling** - Custom scheduling techniques using named barriers.
8. **Arithmetic Intensity** -

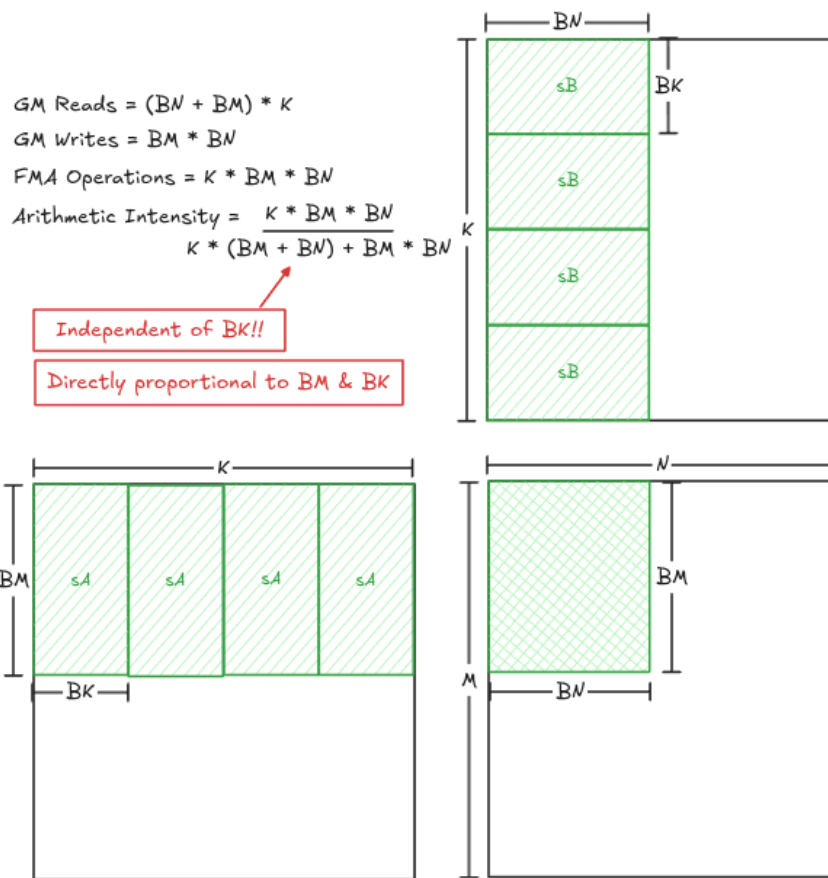


Figure 1: Arithmetic intensity vs Tiling illustration for GEMM (tile size visualization).

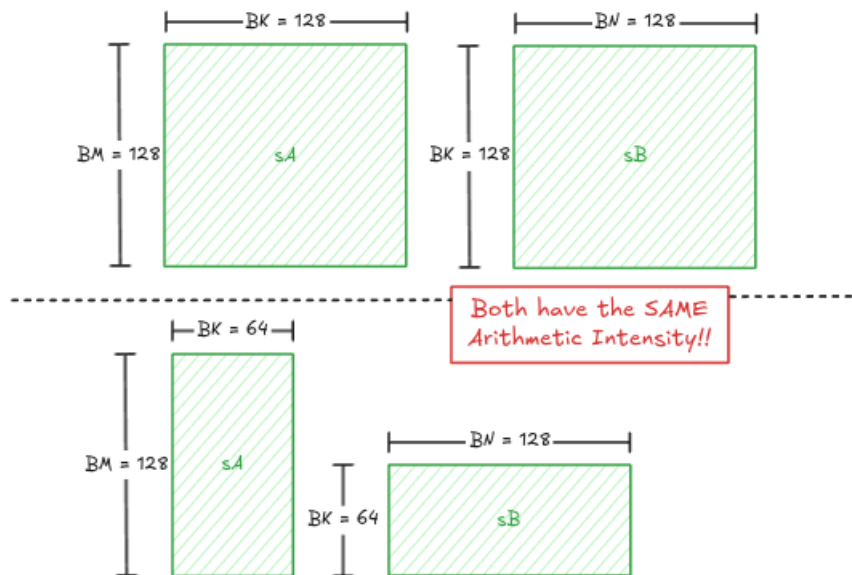


Figure 2: Using rectangular tiles to save shared memory

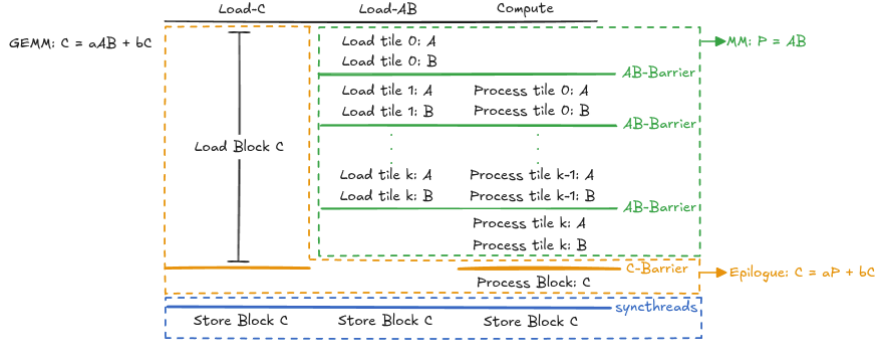


Figure 3: Custom Scheduling Technique for Epilogue Optimization

9. Performance analysis - (All the numbers are in μ s)

Matrix Size	Kernel	Block Size (Arithmetic intensity)			
		128	64	32	16
2048	Baseline	5224.50	5346.43	3936.40	5304.43
	Optimal	1596.94	1337.87	2071.75	4221.77
	Double Buff.	2585.16	3194.23	3931.40	8034.70
	Warp Spec.	2469.26	2334.54	3864.13	7933.93
	Cuda DMA	2093.70	1940.62	3114.32	6375.70
	Double Buff. Async	2405.33	2495.01	3584.37	7301.63
	Warp Spec. Async	2316.18	2149.54	3487.67	7108.50
	Custom Scheduling	—	—	3474.03	—
4096	Baseline	38972.70	49322.70	31060.80	40339.70
	Optimal	11188.90	10149.10	19327.20	85282.30
	Double Buff.	18941.90	27803.30	30988.30	79999.30
	Warp Spec.	17881.10	17778.80	30477.90	64660.00
	Cuda DMA	14758.70	14456.80	26512.00	54924.00
	Double Buff. Async	17122.70	18772.50	28058.40	61825.00
	Warp Spec. Async	16476.10	16327.70	27522.20	62683.00
	Custom Scheduling	—	—	38580.00	—

Table 1: Comparison of GEMM runtimes for various kernel optimizations across tile/block sizes.

Matrix Size	Baseline	Optimal	Double Buffer	Double Buffer Async	Warp Spec.	Warp Spec. Async	Cuda DMA
2048	5286.53	2063.85	3082.16	2817.84	2851.01	2435.04	2112.56
4096	39048.30	14986.70	25340.80	20226.70	27619.90	17464.60	14856.80

Table 2: Comparison of runtimes for different optimization strategies.

Matrix Size	Kernel	Block Size (Arithmetic intensity)	
		128	64
2048	Baseline	758.0	1446.0
	Double Buff.	704.0	1311.0
	Warp Spec. Single Async	2673.0	1041.0
	Warp Spec. Double Async	2549.0	938.0

Table 3: Runtimes for Tensor cores.

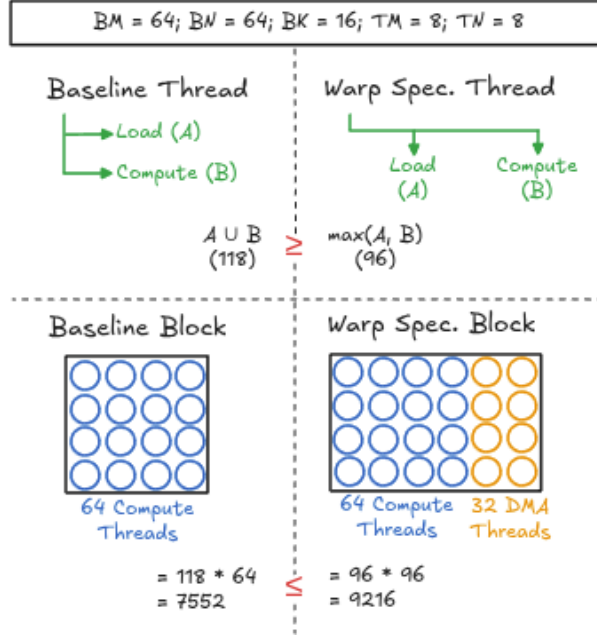


Figure 4: Effect of register pressure on warp specialization

3 Case study - Stencil Kernel

1. The Operation -

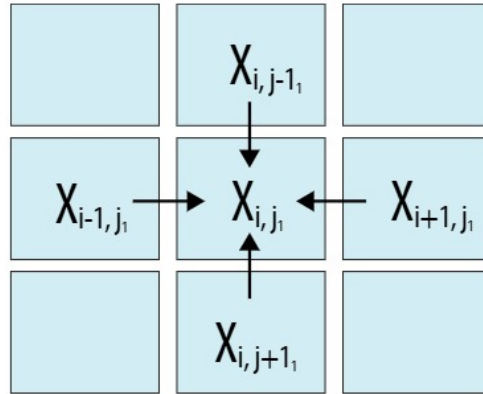


Figure 5: 5 point stencil

2. Shared Memory Optimization & Double buffering -

Different Tile Dimensions

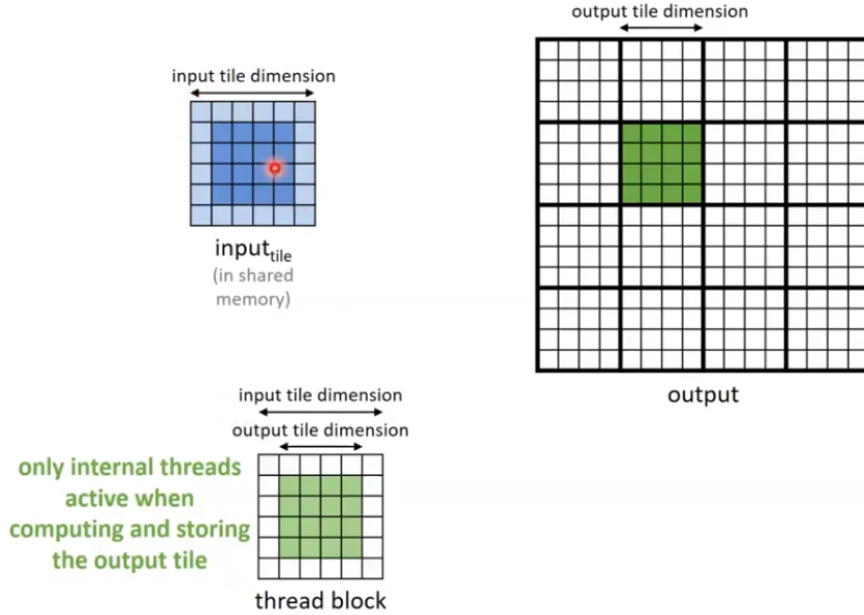


Figure 6: Tiling Stencil

3. Warp Specialization -

```

1  for (int idx = threadIdx.x; idx < total_tile_elements; idx += COMPUTE_THREADS_PER_CTA)
2  {
3      int ii = idx / CUDADMA_TILE_X;
4      int jj = idx % CUDADMA_TILE_X;
5      int gi = start_i + ii;
6      int gj = start_j + jj;
7
8      if (gi >= 0 && gi < n && gj >= 0 && gj < n)
9      {
10         // Use cp.async.ca to copy 4 bytes (1 float) with cache-all policy
11         // This overlaps memory transfer with other operations
12         asm volatile("cp.async.ca.shared.global [%0], [%1], 4;\n"
13                     :
14                     : "r"((unsigned)__cvta_generic_to_shared(&tile[ii][jj])),
15                     "l"(&A[gi * n + gj]));
16     }
17     else
18     {
19         tile[ii][jj] = 0.0f;
20     }
21 }

```

Listing 1: Load warps for Stencil

4. Performance numbers -

Matrix Size	Thread Config	Kernel	Baseline	Double Buffer	Warp Spec
4096	16x16	Time (μ s)	88460.0	88238.0	96136.0
	32x16	Time (μ s)	88403.0	88031.0	89434.0
	32x32	Time (μ s)	86953.0	89230.0	112229.0
	64x16	Time (μ s)	85807.0	88052.0	116647.0

Table 4: Runtime comparison for matrix size **4096** across different thread configurations.

5. Performance analysis -

4 Case Study - Convolution kernel

1. Performance numbers -

Thread Config	Kernel	3D Convolution runtimes (Tensor: $768 \times 768 \times 768$)		
		Baseline (μs)	Double Buffer (μs)	Warp Specialization (μs)
16x16	Time (μs)	11646.00	16580.00	12875.00
32x16	Time (μs)	11598.00	16532.00	14520.00
32x32	Time (μs)	14096.00	21585.00	21334.00
64x16	Time (μs)	13537.00	18992.00	21027.00

Table 5: 3D convolution kernel runtimes for a tensor of size $768 \times 768 \times 768$.

5 Conclusion & Future Work

Abstract—Modern GPUs feature specialized hardware units that enable high-performance, asynchronous dataflow execution. However, the conventional SIMT programming model is fundamentally misaligned with this task-parallel hardware, creating a significant programmability gap. While hardware-level warp specialization is the key to unlocking peak performance, it forces developers to manually orchestrate complex, low-level communication and software pipelines—a process that is labor-intensive, error-prone, and unsustainable. To address this challenge, we present Tawa, an automated compiler that systematically generates high-performance, warp-specialized code from a high-level, tile-based program. Central to our approach is a novel IR abstraction, asynchronous references (`aref`), which expresses warp-level communication without exposing low-level hardware details. Using this abstraction, Tawa automatically partitions programs into producer-consumer roles and manages the intricate dataflow pipeline, relieving developers of invasive kernel rewriting. Evaluation on NVIDIA H100 GPUs across representative LLM kernels shows that Tawa delivers high hardware utilization, achieving up to $1.1\times$ speedup over highly optimized cuBLAS GEMM kernels. For attention workloads, Tawa attains $1.2\times$ speedup over Triton and matches the performance of the hand-optimized CUTLASS C++ FlashAttention-3 kernel with far less programming effort.

Figure 7: Hardware Support Required to mitigate Data Transfer Issues.

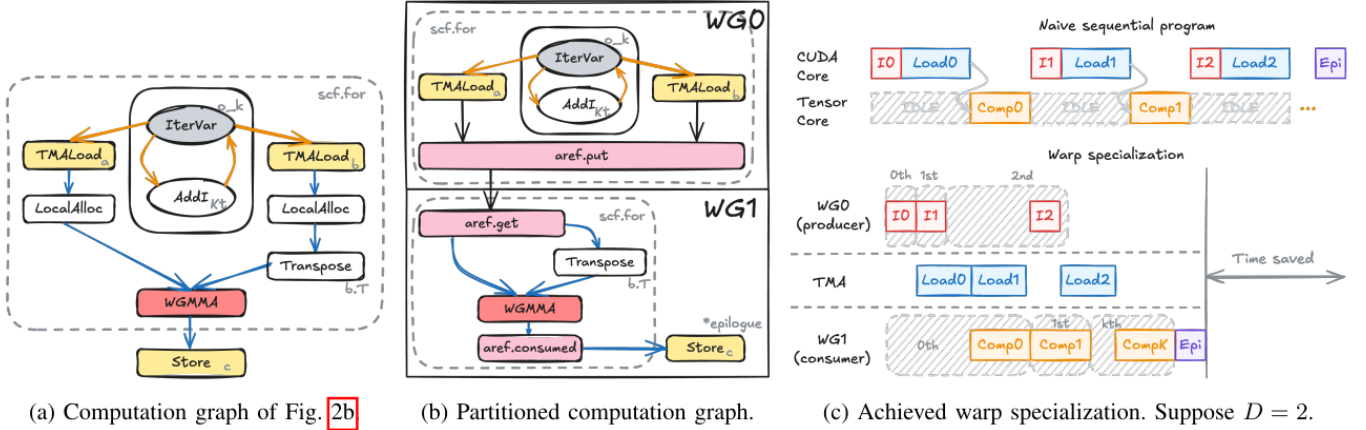


Figure 8: Task-aware partition for the GEMM kernel in Fig. 2b. Notice warp specialisation is just one way of achieving the overlapping in (c), and the actual latency is not shown to scale.

Appendix

A NVCC glitch

A.1 cudaDMA Illegal Instruction Issue

A.1.1 Problem Description

The `jacobi2D.cudaDMA` kernel encounters an illegal instruction error when compiled with optimizations (`-O2/-O3`), but runs without errors when compiled with debug flags (`-g -G`). However, a critical observation reveals suspicious behavior even in the debug build:

- **No illegal instruction error** in debug mode
- **No CPU-GPU mismatch** reported (results appear correct)
- **Print statements inside kernel do NOT appear** in console output

This suggests the kernel may not be executing at all, yet falsely reports success.

A.1.2 Root Causes

1. **Architecture mismatch:** The `cudaMAStrided` helper was written targetting older compute capabilities.
2. **Silent kernel launch failure:** In debug builds, the kernel may fail to launch but errors are not checked, while pre-initialized output buffers contain values matching CPU results.

A.1.3 Diagnostic Steps Performed

- Verified behavioral difference between debug (`-g -G`) and release (`-O3`) builds.
- Observed missing kernel `printf` output despite “successful” execution
- Identified need for explicit error checking after kernel launch.